PDF 4 — Khung chat 03/11 tối → 05/11/2026: v22 → v28, đồng thuận ánh xạ, kiểm tra mới, bài học

Đọc sau `READING_GUIDE.md`, `handoff/STATUS.md`. PDF 1–3 vẫn đúng cho phần trước 03/11; file này ghi phần mới: điểm portal, phát hiện đúng / sai, kinh nghiệm code + vận hành, việc dở dang. Số liệu chi tiết: `EXPERIMENT_LOG.md` entry 14–19.

1. Điểm portal và cách đọc chúng

File nộp	Portal	Ghi chú
v22	0.71	
v23	0.87	**không so được với các điểm sau** (xem dưới)
v24	0.747	dòng Voucher / Phí ship dev_013 bị bỏ khỏi doanh thu (sai)
v25	0.825	bỏ phiếu 5 lần chạy; train 0.9893
v26	0.825	đồng thuận ánh xạ lần 1
X1 / X2 / X3	0.793 / 0.756 / 0.768	v23 + đáp án v26 của một nhóm luật
v27 (bản thay thế), v28	**chưa có điểm**	36 / 37 và 37 / 37 câu kiểm

- **X2 = v23 + đúng 6 câu đổi được 0.756** trong khi v23 từng được 0.87: 6 câu chỉ đổi được ±0.012 → đáp án / tập chấm của portal đã đổi sau lần chấm v23. Giải hệ (các nhóm rời nhau, phủ đủ 91 câu khác nhau): **v23 theo đáp án hiện tại ≈ 0.75**; nhóm X1 (dev_004 ngày phải trả + giao nhiều đợt, dev_008 nợ trong ghi chú, dev_013 voucher / ship / điểm) **+0.04**, câu chủ quan → unanswerable **+0.005**, toolkit điện thoại / sửa xe **+0.018**. Mọi nhóm luật từ v24 đều có lợi.
- Bài học: **chỉ so điểm của các file chấm cùng đợt**; khi điểm mâu thuẫn số học → nghi đáp án / tập chấm đã đổi, kiểm bằng một file lai nhỏ (như X2) trước khi lùi code.

2. Phát hiện đúng (giữ lại)

1. **Đồng thuận ánh xạ (`src/consensus.py`)**: gộp ứng viên ánh xạ của mọi lần chạy, chọn **một** ánh xạ / DN bằng kiểm tra mới nhất, chạy lại **mọi chương trình đã ghi** trên ánh xạ đó, bỏ phiếu lại. Gỡ "may rủi ánh xạ" giữa các lần chạy. Cần thêm: (a) **slot theo chủ đề** — câu nợ / trả góp / xe / thợ dùng ứng viên không có cảnh báo nặng của chủ đề đó, xếp theo khả năng trả lời câu của chủ đề; không có → abstain; (b) xếp hạng chung bỏ điểm phạt chủ đề, lỗi câu chủ đề không trừ điểm chung, đồng thuận chỉ tính giữa ứng viên hợp lý; (c) dự phòng đi tới 4 slot; câu mà 2 ánh xạ đầu không trả lời được thì thêm ứng viên hợp lý có chương trình luật trả lời được; (d) chương trình trùng giữa các lần chạy chạy 1 lần (−38 % thời gian). 8 lần chạy: ~85 phút CPU (3 worker).
2. **Kiểm tra ánh xạ mới (canon.secondary_checks, điểm phạt)** — mỗi cái sinh từ một lỗi thật, không cái nào bắn sai trên 36 ánh xạ tham chiếu train: nợ trên > 25 % số đơn (20); ghi chú "đã trả đủ" mà C còn nợ (10); sổ nợ không đọc được / còn nợ mà C không nợ (10); đơn trả / huỷ trong ghi chú mà C coi hợp lệ (20); biển số bị bỏ / không khớp sổ (20); dòng xe bị bỏ (10); dòng hàng mất tên (20); loại máy mới / cũ / phụ kiện lệch bảng tra (20); giá vốn = 0 mọi hàng (10); cột đơn một giá trị cho mọi đơn (20); sổ ngành (góp, bảo hành, thu máy cũ) có mà C thiếu (20). Phát hiện cột mã đơn trong bảng thô phải có nhiều giá trị khác nhau (cột SL 1.0 / 2.0 từng "khớp" mã đơn 1.0 / 2.0).
3. **Lớp làm sạch / toolkit (tổng quát)**: tên viết tắt khác hoa / thường / dấu → tên đầy đủ của bảng tra (`canon.resolve_short_names`); `paid_at_purchase` + các lần trả sau; tồn kho cần giá vốn thật; cột bảo hành toàn False coi như không có; ngày trả đọc mềm; `weekday_avg` nhận "thứ 3".
4. **Đọc câu**: dòng xe / hãng máy nêu trong câu luôn giữ bộ lọc (`qparse.named_vehicle`); "thứ 3" = thứ Ba = weekday 1 (`qparse.named_weekdays`, chương trình dùng sai thứ không được bỏ phiếu — pipeline lẫn consensus).
5. **Kiểm bằng dữ liệu thô** (tự tính lại câu từ sổ gốc theo rulebook): tìm ra hầu hết lỗi ở trên. Tập 37 câu đã kiểm (`goldkey/devset.py`) dùng như dev set: v25 19, v26 ~15, v27 36, **v28 37 / 37**. Không dùng để chọn đáp án.

6. **File chẩn đoán trên portal** (lai theo nhóm luật / nhóm DN) + số học PA = (đúng − sai) / N cho biết nhóm nào đúng / sai — rẻ hơn nhiều so với đoán. G1–G5 (v27 với một nhóm DN để abstain) đã tạo, chưa nộp.
7. Train cùng code mới: v25 0.9893, **v28 0.9897** (2977 / 8 / 15); oracle 2967 / 1 / 32 không đổi.

3. Phát hiện sai / sai lầm (tránh lặp lại)

1. Tin v23 = 0.87 là mốc tuyệt đối → mất một vòng phân tích "vì sao v25 thua". Xem §1.
2. v24 bỏ dòng Voucher / Dùng điểm / Phí ship khỏi doanh thu dev_013 (theo §1 rulebook) → portal tụt; giữ trong doanh thu (`--adj-rows keep`, mặc định) là đúng với đáp án BTC.
3. Tự tính tay sai: sổ ghi STT ở dòng đầu đơn, các dòng sau để trống (§8 rulebook) → phải **ffill** trước khi gom đơn; quên → kết luận sai rằng ánh xạ đếm thừa (dev_011). Luôn ffill khoá đơn / ngày / khách / ghi chú khi tự kiểm.
4. Đồng thuận chọn một ánh xạ cho cả DN: thêm điểm phạt mới có thể **đẩy cả DN sang ánh xạ tệ hơn** ở phần khác (dev_014 sang ánh xạ không có trạng thái trả hàng; v26 sang ánh xạ không có loại máy). Giải pháp: slot theo chủ đề + kiểm tra cho mọi trường cốt lõi. Mỗi lần đổi xếp hạng phải chạy lại dev set (v27 lần đầu làm hỏng 6 câu ngày phải trả dev_004).
5. Nhiều bản sao (từ nhiều lần chạy) của một ánh xạ lỗi đồng thuận với nhau và thắng ánh xạ sạch → chỉ tính đồng thuận giữa ứng viên không có cảnh báo nặng.
6. Một ngoại lệ trong kiểm tra (dòng tiêu đề "Ngày trả" lẫn vào ngày trả) làm kiểm tra **im lặng** → ánh xạ lỗi không bị phạt. Kiểm tra phải đọc dữ liệu mềm (`errors="coerce"`).
7. Ước lượng sai số câu sai bằng giả định chấm toàn bộ 1000 câu cố định — đúng về số học, nhưng giả định "đáp án không đổi" sai (§1).

4. Kinh nghiệm vận hành

- **Quota Kaggle**: tuần W45 (đến hết CN 08/11) **mọi acc đã hết 30 h GPU** (kể cả loinguyentran, anderson1202, phatle1578 sau v25 + v28). `~/.cache/zc_quota.json` ghi acc hết quota theo tuần ISO. Một bản thử đầy đủ = 6 kernel × ~1 h. Muốn chạy thêm tuần này → xin user acc mới (đã accept rules `gemma-4-developer-agent`).
- `fanout.py` tự chuyển kernel push hỏng do quota sang acc khác (tối đa 2 / acc); vẫn có thể hỏng hết khi mọi acc cạn.
- **Container khởi động lại** làm mất tiến trình nền (một lần mất bước consensus đang chạy) → chạy lại; kết quả trong scratch vẫn còn. Đĩa có giới hạn (~30 GB trống): xoá `cons_*/canon` (2–4 GB mỗi lần chạy) và thư mục nghiên cứu cũ.
- Không chạy test với đường dẫn cache train làm thư mục scratch (test_pipeline ghi thư mục con vào đó). `run_all.sh` tự dùng thư mục riêng: `CACHE=<cache_train> SCRATCH=<dir> bash tests/run_all.sh`.
- Mỗi lần sửa code trong lúc consensus đang chạy: tiến trình dùng code cũ → huỷ (theo PID) và chạy lại.
- Oracle đúng lệnh: `python3 tests/eval_lib.py data/raw/train/train <cache_train> --show 0 --clean question_cleaning/train_questions_clean.jsonl` → phải in `TOTAL {'ok': 2967, 'unparsed': 32, 'wrong': 1}`.

5. Bản đồ code mới

File	Nội dung mới
`src/consensus.py`	đồng thuận ánh xạ + slot chủ đề (`TOPIC_FN`, `TOPIC_WARN`, `question_topics`, `topic_bad`, `general_score`, `rank`), ghi `work/pool.json` (xếp hạng ứng viên)
`src/canon.py`	các kiểm tra §2.2, `OWING_MAX`, `_order_key_col`, `resolve_short_names`, `warning_penalty`
`src/dm.py`	`paid_until` (trả lúc mua + trả sau), `stock_value` cần giá vốn, `weekday_avg("thứ 3")`, `_order_adjustments`
`src/industry.py`	toolkit ngành điện thoại / sửa xe / mẹ và bé; `_is_warranty_ticket`
`src/qparse.py`	`named_vehicle`, `named_weekdays`, `weekday_consistent`, mẫu ngành

File	Nội dung mới
`automation/jobs/mksub5.sh`	bỏ phiếu 5 lần chạy + consensus → `submissions/`
`automation/jobs/v25.jobs`, `v28.jobs`	job chuẩn (train + 5 public, seed khác nhau)
`automation/jobs/analysis/`	công cụ so sánh / kiểm thô (chuyển từ scratch): `cmp.py`, `diff_detail.py`, `warnprobe.py`, `revprobe.py`, `debtprobe.py`, `phone_direct.py`, `dev014_direct.py`
`goldkey/`	dev set đã kiểm + bộ đáp án tham chiếu đang làm (chỉ để đánh giá, **không** dùng trong `src/`)
`docker/predict.sh`	`CONSENSUS=1` tuỳ chọn (mặc định tắt)

6. Việc dở dang / tiếp theo

1. **Chờ điểm portal v28** (và v27 bản thay thế, G1–G5 nếu user nộp). Nếu v28 > v25 → bật `CONSENSUS=1` mặc định trong `docker/predict.sh` (đo thời gian: 3 lần chạy + consensus ≤ 6 h trên 2×RTX 5090; consensus 5 lần chạy ~35 phút CPU 3 worker) và cân nhắc RUNS=5.
2. **Bộ đáp án tham chiếu** `goldkey/` (user yêu cầu 05/11): đáp án từng câu public do Claude tự tính từ dữ liệu thô theo rulebook, kèm mức tin cậy; dùng để chấm offline các bản sau. Tiến độ ở `goldkey/README.md`.
3. Còn ~80 câu sai (ước lượng từ v26) — tìm bằng G1–G5 hoặc so với `goldkey/`, sửa bằng quy tắc chung, chạy lại consensus (không cần GPU).
4. Docker cuối trước **16/11/2026 23:59 GMT+7**: build, đo 2×RTX 5090, kiểm $|\Delta PA| \leq 0.03$ giữa 2 lần build.